MEMORIA DEL PROYECTO INDIVIDUAL: DEAD SOULS

Módulo: Programación (0485) — 1.º DAW
Curso Académico: 2026/2027
Entrega: Primera Evaluación — Entrega 1 (Videojuego de consola en Java)
Alumno: Jorge Fernández Redondo
URL de mi GitHub: https://github.com/jfdez828/DeadSouls

1. DESCRIPCIÓN DEL JUEGO Y REGLAS DE PARTIDA

1.1. Temática y Objetivo

He programado **Dead Souls**, un videojuego de rol (RPG) que funciona por la consola de comandos. El argumento consiste en controlar a un aventurero en un reino hostil. El objetivo principal de la partida es **derrotar a un total de 10 enemigos comunes** para limpiar la zona. En cuanto se consigue esta cifra, el cielo se vuelve negro y aparece el jefe final: **El Señor de las Sombras**. La victoria absoluta se alcanza al vencer a este Boss en el combate definitivo, mientras que la partida termina en derrota si la salud del personaje llega a cero.

1.2. Las Reglas y Mecánicas del Juego

Para garantizar la coherencia del juego y evitar fallos lógicos, he establecido las siguientes normas mediante el código:

- **Creación del Personaje:** Al empezar, el programa solicita el nombre del usuario y otorga **10 puntos de estadísticas** iniciales. Estos puntos se pueden repartir libremente en un menú interactivo para mejorar la Vida Maxima (+10 PV por punto), el Ataque (+2 ATQ) o la Defensa (+1 DEF).
- **Sistema de Experiencia:** Al derrotar enemigos comunes, el jugador consigue puntos de experiencia aleatorios. Al acumular **100 puntos de EXP**, el personaje sube de nivel de forma automática, lo que restablece su salud actual al nuevo máximo y aumenta sus atributos de combate de forma permanente.
- **Combate por Turnos:** Durante la exploración, el código selecciona un monstruo al azar (un Duende, un Lobo Salvaje, un Esqueleto o un Orco). Las peleas se desarrollan por turnos estrictos. En el turno del jugador, se puede elegir entre realizar un ataque físico (el daño base más una variación aleatoria), defenderse (para duplicar la defensa durante ese turno) o consumir una poción curativa del inventario. Después, el enemigo ejecuta su ataque restando el valor de la defensa del héroe.

- **Penalización por Derrota:** Si un enemigo común derrota al jugador, unos aldeanos lo rescatan a cambio de una penalización: **se pierde el 50% de las monedas** acumuladas y la salud se restablece solo a la mitad de su capacidad máxima. Si la derrota ocurre contra el Boss Final, se aplica una regla de regresión: el contador de enemigos derrotados **se reinicia a 5**, obligando al jugador a explorar y farmear de nuevo antes de poder volver a desafiarlo.

2. FUNCIONALIDADES IMPLEMENTADAS

El menú principal está controlado por un bucle `while` y una estructura `switch` que permiten al jugador elegir de forma repetida entre **4 acciones operativas principales**:

- **Opción 1: Explorar el Reino:** Controla la selección aleatoria de los enemigos y el bucle interno del combate por turnos. Esta acción modifica directamente la salud, las monedas, la experiencia y el contador de bajas de la partida.
- **Opción 2: Consultar Estadísticas:** Muestra por pantalla los datos actualizados del personaje: el nivel, la experiencia acumulada y la necesaria para el siguiente nivel, los puntos de vida actuales respecto al máximo, el ataque, la defensa y el total de monedas.
- **Opción 3: Ver Inventario / Usar Objeto:** Muestra las 5 posiciones del vector de inventario. Si se selecciona una poción de vida, el programa comprueba que la salud no esté ya al máximo para no malgastarla. Al usarla, se recuperan hasta 40 PV y esa posición del array pasa a estar `"Vacio"`.
- **Opción 4: Visitar la Tienda:** Abre un submenú para realizar transacciones comerciales. Permite gastar las monedas ganadas en los combates para comprar pociones (si hay hueco en el inventario) o mejorar de forma permanente el arma y el escudo del héroe para subir el ataque y la defensa.

3. DISEÑO TÉCNICO Y ARQUITECTURA DEL CÓDIGO

3.1. Variables y Constantes

He utilizado la convención *camelCase* para las variables y he definido las constantes obligatorias del PDF mediante la palabra clave `final` en MAYÚSCULAS al principio de la clase para que el código sea limpio y estructurado.

Nombre en el código	Tipo	Función en el flujo
ENEMIGOS_PARA_BOSS	int (Constante)	Número fijo (10) de victorias necesarias para invocar al jefe.

CURACION_POCION	int (Constante)	Puntos fijos (40) de salud que restablece la poción.
PRECIO_POCION, PRECIO_DAGAS...	int (Constantes)	Costes fijos de los artículos de la tienda.
vidaActual y vidaMaxima	int (Variable)	Controlan la salud del héroe en todo momento.
ataqueHeroe y defensaHeroe	int (Variable)	Atributos de combate que aumentan en la tienda o al subir de nivel.
monedas	int (Variable)	Monedero del jugador. Aumenta con los botines y disminuye con las compras.
enemigosDerrotados	int (Variable)	Contador incremental de victorias que determina cuándo aparece el Boss.

3.2. Estructuras de Control

He aplicado los conceptos de las unidades didácticas del curso para gestionar las decisiones y repeticiones:

- **Bucles (`while`):** Utilizo `while (jugando)` para mantener el juego activo hasta que se elija la opción de salir. También uso un `while` en el reparto de puntos inicial (no permite avanzar hasta gastar los 10 puntos de creación) y un `while (enCombate)` para la secuencia de turnos de la pelea.
- **Condicionales (`if-else` y `switch`):** El menú principal y la tienda funcionan con estructuras `switch`. Los condicionales `if` controlan las reglas del juego, como verificar si el jugador tiene saldo suficiente para comprar o si los puntos de vida de un enemigo han llegado a cero.
- **Sentencias de Salto:** Utilizo `break` para romper los bucles de combate o salir de la tienda de forma inmediata. Uso `continue` en el inventario rápido de combate; si el jugador decide cancelar el uso de un objeto, el programa interrumpe esa acción y vuelve al principio del turno de selección sin que el enemigo ataque.

3.3. Uso de Arrays Unidimensionales

Al no utilizar todavía programación orientada a objetos en esta primera evaluación, he implementado dos soluciones basadas en arrays unidimensionales:

1. **Vector de Inventario (`String[]`):** Es un array de tamaño fijo con 5 elementos. Controla los objetos iniciales y las ranuras libres mediante el literal `"Vacio"`. Cuando se compra en la tienda, un bucle `for` recorre el array buscando la primera posición que contenga `"Vacio"` para guardar el nuevo artículo.
2. **Arrays Paralelos para la Base de Datos:** He enlazado la información de los monstruos utilizando tres arrays independientes que comparten el mismo índice numérico:

a. String[] enemigosNombres = { "Duende", "Lobo Salvaje", "Esqueleto", "Orco" };
b. int[] enemigosVidaMax = { 30, 45, 60, 80 };
c. int[] enemigosAtaque = { 8, 12, 15, 20 };

Generando un número aleatorio con random.nextInt(enemigosNombres.length) obtengo un índice (por ejemplo, el 2). De esta forma, el programa sabe síncronamente que el enemigo es el Esqueleto, que tiene 60 de vida y 15 de fuerza de ataque.

4. PLAN DE PRUEBAS Y CASOS LÍMITE

Para asegurar que el programa es estable frente a errores del usuario, he realizado las siguientes comprobaciones por la consola:

- **Prueba 1: Introducción de letras en menús numéricos.** Si el usuario introduce una cadena de texto como "error" cuando se pide un número, el programa fallaría con una excepción. Para evitarlo, he implementado el método leerEntero(). El condicional !scanner.hasNextInt() detecta que la entrada no es un entero, limpia el buffer de la consola y solicita el dato de nuevo sin detener la ejecución del juego.
- **Prueba 2: Compras sin saldo suficiente.** Accedí a la tienda con 50 monedas e intenté comprar dos escudos de 45 monedas. La primera transacción se completó dejando el saldo en 5 monedas. En la segunda compra, el condicional if (monedas >= PRECIO_ESCUDO) denegó la operación al comprobar que 5 es menor que 45, mostrando el mensaje "No tienes suficientes monedas.".
- **Prueba 3: Desbordamiento del inventario.** Compré varios objetos hasta ocupar las 5 ranuras del array e intenté adquirir una poción más. El bucle for revisó los 5 elementos del vector, y al no encontrar ninguna coincidencia con la palabra "Vacio", bloqueó la compra y notificó que no había espacio disponible.

5. DIFICULTADES ENCONTRADAS Y MEJORAS FUTURAS

5.1. Problemas Técnicas y Soluciones Aplicadas

- **Gestión del buffer del Scanner:** Al principio, tras introducir un número en el menú, el programa se saltaba la siguiente lectura de texto donde el jugador debe escribir el nombre de su héroe. Esto ocurre porque nextInt() no procesa el salto de línea (\n) y lo deja en el buffer. Lo solucioné añadiendo una llamada a scanner.nextLine() vacía inmediatamente después de leer cualquier número para limpiar el buffer antes de la siguiente captura.
- **Cálculo de daños negativos en combate:** Al restar la defensa del héroe al ataque de un enemigo, si la defensa era muy alta el resultado daba un número negativo. Al

aplicar la operación vidaActual -= daño, restar un valor negativo provocaba que el ataque del enemigo terminara curando al personaje. Lo arreglé utilizando la función matemática Math.max(1, dañoCalculado), asegurando que cualquier ataque inflija al menos 1 punto de daño neto.

5.2. Ampliaciones para la Segunda Evaluación

Como el proyecto es incremental, tengo planificadas las siguientes mejoras para las próximas entregas:

1. **Migración a Programación Orientada a Objetos (POO):** Eliminar los arrays paralelos y organizar el diseño creando clases reales (Heroe, Enemigo, Objeto) con sus propios constructores y métodos encapsulados.
2. **Uso de Estructuras Dinámicas (ArrayList):** Sustituir el array de inventario de tamaño fijo por una colección de tipo ArrayList. Esto permitirá gestionar la inserción y el descarte de objetos de forma mucho más ágil y sin un límite rígido de 5 posiciones.
3. **Persistencia en Ficheros:** Implementar la lectura y escritura de archivos de texto (.txt) para poder guardar el progreso de las estadísticas y las monedas del jugador en el disco duro, permitiendo reanudar la partida más adelante